SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1701**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 3 **Duration: 1 Hour**
Total Marks:30

Annexure A

Dry Run Evaluation

Code of Conduct:

I GIFTSON KEVIN MATTHEW.Y (192511167) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

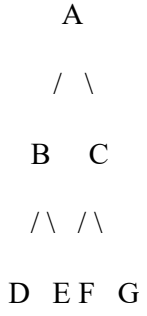
Signature of the student: GIFTSON KEVIN MATTHEW.Y

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



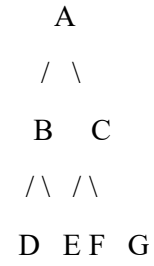
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**

   1 3 6
   5 0 2
   4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Question 1: Breadth-First Search (BFS)

Given graph

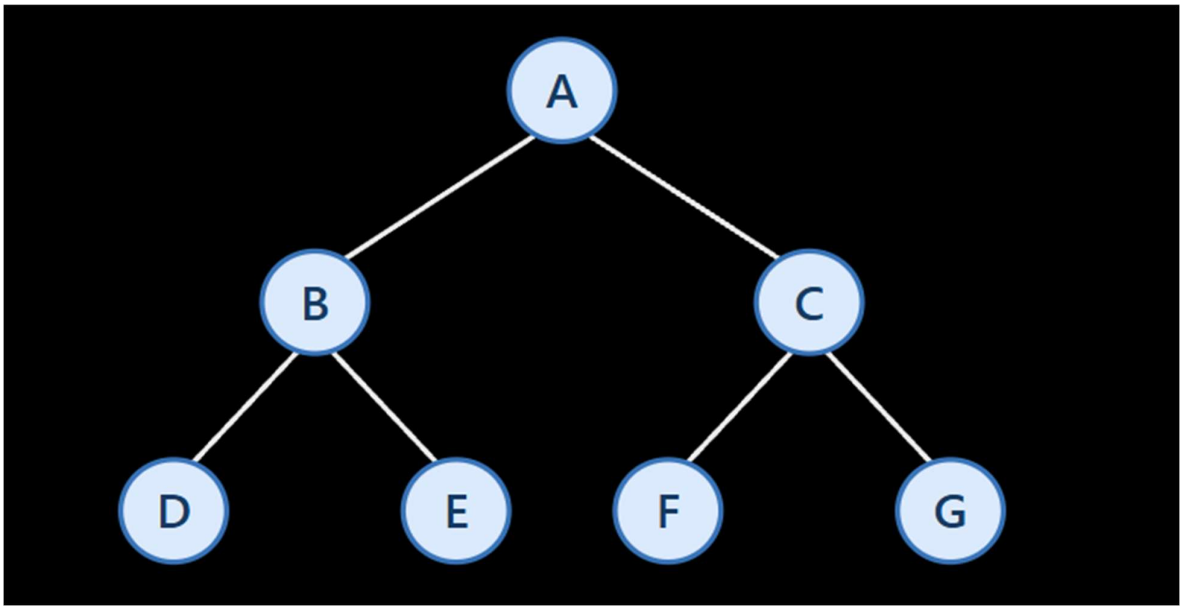


Figure 1: Given tree for BFS and DFS

Concept of BFS

Breadth-First Search (BFS) is an uninformed search algorithm that explores nodes level by level. It visits the root node first, then all nodes at the next level from left to right, and continues until every required node is visited.

BFS uses a FIFO (First In, First Out) queue. The node inserted first is removed and processed first.

Algorithm:

1. Insert the starting node A into the queue.
2. Remove the front node from the queue and mark it as visited.
3. Insert its unvisited children from left to right.
4. Repeat until the queue becomes empty.

Dry-run table

Iteration	Queue before expansion	Visited node	Queue after expansion
1	A	A	B, C
2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G

5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	Empty

Queue entries are shown from front to rear.

Answers

1. Write the BFS traversal order.

The traversal order is:

[boxed{A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G}]

2. Show the queue contents after each iteration.

- After iteration 1: B, C
- After iteration 2: C, D, E
- After iteration 3: D, E, F, G
- After iteration 4: E, F, G
- After iteration 5: F, G
- After iteration 6: G
- After iteration 7: Empty

Explanation: BFS visits A first, then its children B and C. Next, it visits the children of B and C in level order. Therefore, the algorithm explores all nodes at one depth before moving to the next depth.

Question 2: Depth-First Search (DFS)

Concept of DFS

Depth-First Search (DFS) is an uninformed search algorithm that explores one branch as deeply as possible before returning to explore another branch.

DFS commonly uses a LIFO (Last In, First Out) stack. The most recently inserted node is processed first.

Algorithm

1. Start at node A.
2. Visit A and move to its leftmost unvisited child, B.
3. Continue to the deepest node, D.
4. If no unvisited child exists, backtrack to the previous node.
5. Continue with E, then C, followed by F and G.

Dry-run table

Assuming left-to-right traversal:

Iteration	Current node	Action	Visited order
1	A	Visit A; move to B	A
2	B	Visit B; move to D	A, B
3	D	Visit leaf D; backtrack	A, B, D
4	E	Visit E; backtrack to A	A, B, D, E
5	C	Visit C; move to F	A, B, D, E, C
6	F	Visit leaf F; backtrack	A, B, D, E, C, F
7	G	Visit leaf G	A, B, D, E, C, F, G

Final answers

1. DFS traversal order

[boxed{A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G}]

2. Data structure used: Stack (LIFO).

Explanation: DFS explores the left branch A → B → D before backtracking to visit E. It then explores the right branch A → C → F → G. Unlike BFS, DFS does not finish an entire level before moving deeper.

BFS vs DFS

Feature	BFS	DFS
Search method	Level by level	Depth by depth
Data structure	Queue (FIFO)	Stack (LIFO)
Traversal	A, B, C, D, E, F, G	A, B, D, E, C, F, G
Shortest path	Guaranteed for unweighted graphs	Not guaranteed
Memory	Can use more memory for wide trees	Often uses less memory on wide trees

Question 3: Solve the 8-Puzzle Using BFS

Given initial and goal states

Initial state:

1	3	6
5	□	2
4	7	8

Initial state (0 is the blank tile)

Initial state (0 is the blank tile)

Goal state:

1	2	3
4	5	6
7	8	□

Goal state

Concept of the 8-puzzle problem

The 8-puzzle is a state-space search problem consisting of eight numbered tiles and one blank space on a 3 × 3 board. The blank space can move up, down, left, or right when the move is legal.

In this problem:

- Initial state: The starting arrangement given in the question.
- Goal state: The arrangement in which tiles are ordered from 1 to 8, with the blank at the bottom-right.
- Operators: Move the blank tile up, down, left, or right.
- Search algorithm: BFS.
- Objective: Find a sequence of legal moves from the initial state to the goal state.

BFS stores generated states in a FIFO queue and avoids revisiting states already discovered.

Dry run: first five expansions

For the following trace, states are written row by row. For example, 136/502/478 represents:

1	3	6
5	0	2
4	7	8

The queue contains complete board states, not individual tile numbers. Since the queue can become large, the table gives its size and the newly generated states for each early iteration.

Iteration	Current state expanded	Queue before expansion	Queue after expansion
1	136/502/478	1 state	4 states
2	106/532/478	4 states	5 states
3	136/572/408	5 states	6 states
4	136/052/478	6 states	7 states
5	136/520/478	7 states	8 states

The order of legal moves used here is up, down, left, and right. A state already discovered is not inserted again.

Important: These are only the first five expansions requested in the table. The goal is not reached within these five iterations; BFS must continue processing the queue.

Answers to the four questions

1. Which node or state is expanded in each iteration?

In BFS, the state at the front of the queue is expanded first. The first five expanded states are:

1. 136/502/478
2. 106/532/478
3. 136/572/408
4. 136/052/478
5. 136/520/478

The algorithm continues until the goal state reaches the front of the queue and is removed for expansion.

2. How many states are generated in total?

Using the specified move order (up, down, left, right) and counting each distinct board configuration once:

- Unique states discovered: 500, including the initial state.
- States removed from the queue and expanded: 311, including the goal state.

These counts depend on the move-generation order and on whether the initial state is included in the total.

3. Write the complete solution path from the initial state to the goal state.

The shortest solution contains 8 moves and 9 board configurations.



Initial state



Step 1

Move 1: Blank right



Step 2

Move 2: Blank up



Step 3

Move 3: Blank left



Step 4

Move 4: Blank down



Step 5

Move 5: Blank left



Step 6

Move 6: Blank down



Step 7

Move 7: Blank right



Step 8

Move 8: Blank right

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

BFS explores all states at depth 0, then depth 1, then depth 2, and so on. Each legal move has equal cost, so the first time BFS reaches the goal, it has found a solution with the minimum possible number of moves.

Therefore, BFS guarantees the shortest solution in terms of moves, provided the puzzle is solvable and repeated states are handled correctly.